

Name: Pramod Pandit

Reg no.: 12305901

1. Maximum Depth of Binary Tree

104. Maximum Depth of Binary Tree Solved

Easy Topics Companies

Given the `root` of a binary tree, return its *maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if(!root) return 0;
        if(!root->left) return 1 + maxDepth(root->right);
        if(!root->right) return 1 + maxDepth(root->left);
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

Accepted Runtime: 0 ms

2. Same Tree

100. Same Tree Solved

Easy Topics Companies

Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:

```
bool isSameTree(TreeNode* p, TreeNode* q) {
    if(p==NULL && q==NULL){
        return true;
    }
    if(p==NULL || q==NULL){
        return false;
    }
    if(p->val != q->val){
        return false;
    }
    return isSameTree(p->left, q->left) &&
        isSameTree(p->right, q->right);
}
```

Accepted Runtime: 0 ms

3. Invert Binary Tree

226. Invert Binary Tree

Easy Topics Companies

Given the `root` of a binary tree, invert the tree, and return its *root*.

Example 1:

Input: `root = [4,2,7,1,3,6,9]`

```
TreeNode* invertTree(TreeNode* root) {
    if (root == nullptr)
        return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```

Accepted Runtime: 0 ms

4. Symmetric Tree

Problem List

Description

Editorial

Solutions

Submissions

101. Symmetric Tree

Solved

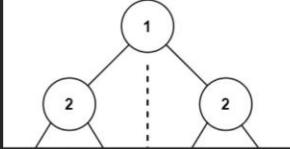
Easy

Topics

Companies

Given the `root` of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Example 1:



16.9K 332

Code

```
1 class Solution {
2 public:
3     bool isMirror(TreeNode* left, TreeNode* right) {
4         if (left == NULL && right == NULL)
5             return true;
6
7         if (left == NULL || right == NULL)
8             return false;
9
10        if (left->val != right->val)
11            return false;
12
13        return isMirror(left->left, right->right) &&
14               isMirror(left->right, right->left);
15    }
16    bool isSymmetric(TreeNode* root) {
17        return isMirror(root->left, root->right);
18    }
19 }
```

Testcase Test Result